

Submission Worksheet

Submission Data

Course: IT202-008-S2026

Assignment: IT202 Milestone 1 - 2026

Student: Ashton T. (ast56)

Status: Submitted | **Worksheet Progress:** 95%

Potential Grade: 9.33/10.00 (93.30%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/13/2026 4:54:23 PM

Updated: 4/13/2026 11:51:34 PM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/ast56>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-1-2026/view/ast56>

Instructions

- Overview Link: <https://youtu.be/IDtqdaLwcVg>

1. Refer to Milestone1 of this doc:

<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWWwfo/view>

2. Ensure you read all instructions and objectives before starting.

3. Ensure you've gone through each lesson related to this Milestone

4. Switch to the Milestone1 branch

1. `git checkout Milestone1` (ensure proper starting branch)
2. `git pull origin Milestone1` (ensure history is up to date)

5. Fill out the below worksheet

- Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
- Ensure proper styling is applied to each page
- Ensure there are no visible technical errors; only user-friendly messages are allowed

6. Once finished, click "Submit and Export"

7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github

1. `git add .`
2. `git commit -m "adding PDF"`
3. `git push origin Milestone1`
4. On Github merge the pull request from Milestone1 to qa
5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the heroku prod links work)

8. Upload the same PDF to Canvas

9. Sync Local

10. `git checkout qa`

11. `git pull origin qa`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

≡ Task #1 (0.67 pts.) - Validation

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

🖼 Part 1:

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<form onSubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email</label>
    <input id="email" type="email" name="email" required value="<?php echo se($_POST, 'username'); ?>" />
  </div>
  <div>
    <label for="username">Username</label>
    <input type="text" name="username" required maxlength="40" value="<?php echo se($_POST, 'username'); ?>" />
  </div>
  <div>
    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <div>
    <label for="confirm">Confirm</label>
    <input type="password" name="confirm" required minlength="8" />
  </div>
  <input type="submit" value="Register" />
</form>
```

Register HTML Validation Code

The screenshot shows a registration form with a blue background. The 'Email' field contains the text 'ash'. Below the form, a red error message box is displayed, stating: 'Please include an '@' in the email address. 'ash' is missing an '@'.'

Confirm

Incorrect format for email

Register

Email

Username

Password

Confirm

Register



Please fill out this field.

Email Field cannot be empty

Password

Confirm



Please fill out this field.

Password field cannot be empty

Username

Password



Please lengthen this text to 8 characters or more (you are currently using 6 characters).

Password Must be 8 Characters long



Saved: 4/13/2026 5:38:55 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation works by using attributes like required and minlength. These let the browser check the user inputs before it is submitted. This makes it so that the user can

browser check the user inputs before it is submitted. This makes it so that the user can catch simple mistakes before an error is pushed.

 Saved: 4/13/2026 5:38:55 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
function validate(form) {
  //username is required, follow the format and length constraints
  let email = form.email.value;
  let username = form.username.value;
  let password = form.password.value;
  let confirm = form.confirm.value;

  let isValid = true;

  if (!isValid) {
    //username is required, follow the format and length constraints
    if (!username) {
      isValid = false;
      document.getElementById("username").innerHTML = "Invalid username";
    }
    //email is required, follow the format and length constraints
    if (!email) {
      isValid = false;
      document.getElementById("email").innerHTML = "Invalid email address";
    }
    //password is required, follow the format and length constraints
    if (!password) {
      isValid = false;
      document.getElementById("password").innerHTML = "Invalid password";
    }
    //confirm is required, follow the format and length constraints
    if (!confirm) {
      isValid = false;
      document.getElementById("confirm").innerHTML = "Invalid confirm";
    }
    //password must be at least 8 characters long
    if (password.length < 8) {
      isValid = false;
      document.getElementById("password").innerHTML = "Password must be at least 8 characters long";
    }
    //passwords must match
    if (password !== confirm) {
      isValid = false;
      document.getElementById("confirm").innerHTML = "Passwords do not match";
    }
  }

  return isValid;
}
```

Register JS Validate

(js) Invalid email address
Register

(js) Invalid username format
Username 212MUJI

(js) Password must be at least 8 characters long
Password *****

(js) Passwords do not match
Confirm *****
Register

Render JS Validate Messages

 Saved: 4/13/2026 5:45:14 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation uses the validate() function to check whether or not the user inputs match the correct parameters for each field such as correct email and username format, password length and whether the password and confirmation fields match. This validation displays colored error messages if any of these parameters are not met and the form is not submitted.



Saved: 4/13/2026 5:45:14 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use

```

1 <?php
2 // ... (previous code) ...
3 // Validation functions
4 function validateEmail($email) {
5     $email = filter_var($email, FILTER_VALIDATE_EMAIL);
6     return $email !== false;
7 }
8
9 function validateUsername($username) {
10    $username = strtolower($username);
11    if (!preg_match('/^[a-z0-9_-]{3,30}$/', $username)) {
12        return false;
13    }
14    return true;
15 }
16
17 function validatePassword($password) {
18    if (strlen($password) < 8) {
19        return false;
20    }
21    if (!preg_match('/[a-zA-Z0-9!@#$%^&*]{8,}$/', $password)) {
22        return false;
23    }
24    return true;
25 }
26
27 function validateConfirmPassword($password, $confirmPassword) {
28    if ($password !== $confirmPassword) {
29        return false;
30    }
31    return true;
32 }
33
34 // ... (validation logic) ...
35
36 // ... (validation logic) ...
37
38 // ... (validation logic) ...
39
40 // ... (validation logic) ...
41
42 // ... (validation logic) ...
43
44 // ... (validation logic) ...
45
46 // ... (validation logic) ...
47
48 // ... (validation logic) ...
49
50 // ... (validation logic) ...
51
52 // ... (validation logic) ...
53
54 // ... (validation logic) ...
55
56 // ... (validation logic) ...
57
58 // ... (validation logic) ...
59
60 // ... (validation logic) ...
61
62 // ... (validation logic) ...
63
64 // ... (validation logic) ...
65
66 // ... (validation logic) ...
67
68 // ... (validation logic) ...
69
70 // ... (validation logic) ...
71
72 // ... (validation logic) ...
73
74 // ... (validation logic) ...
75
76 // ... (validation logic) ...
77
78 // ... (validation logic) ...
79
80 // ... (validation logic) ...
81
82 // ... (validation logic) ...
83
84 // ... (validation logic) ...
85
86 // ... (validation logic) ...
87
88 // ... (validation logic) ...
89
90 // ... (validation logic) ...
91
92 // ... (validation logic) ...
93
94 // ... (validation logic) ...
95
96 // ... (validation logic) ...
97
98 // ... (validation logic) ...
99
100 // ... (validation logic) ...

```

PHP validate Code Snippet

Invalid email address.

Register

Username must be lowercase, alphanumerical, can only contain _ or -, and be between 3 to 30 characters

Username @as

Password

Password must be at least 8 characters long.

Confirm

Passwords must match.

Register

PHP validate messages



Saved: 4/13/2026 6:28:48 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Unlike the other validations, PHP happens on the server side where checks are performed to match user input to the parameters set for each field. When an error occurs flash messages are displayed and the data is not processed ensuring that only verified data is added to the database.



Saved: 4/13/2026 6:28:48 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include the render.com prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1

<https://github.com/PanicGeneral/ast56-it202-008/pull/26>



URL

<https://github.com/PanicGen>



URL #2

<https://ast56-it202-008.onrender.com/project/register.php>



URL

<https://ast56-it202-008.onren>



URL #3

https://github.com/PanicGeneral/ast56-it202-008/Milestone1/public_html/project/register.php



URL

<https://github.com/PanicGen>





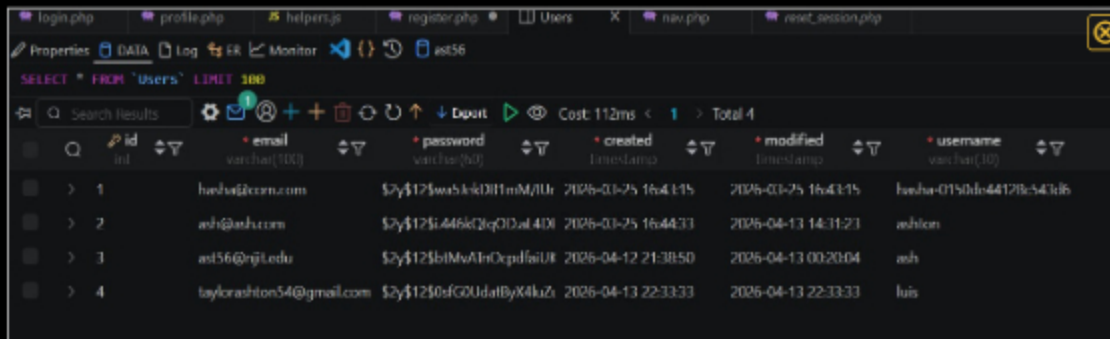
Saved: 4/13/2026 6:32:30 PM

Task #3 (0.67 pts.) - Database - Users table

Progress: 100%

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferably a recent one during evidence capturing)



The screenshot shows the VS Code interface with a database view of the 'Users' table. The table has 4 records. The columns are: id, email, password, created, modified, and username. The data is as follows:

id	email	password	created	modified	username
1	husha@ccm.com	\$2y\$12\$w5b5kD11mM/UU	2026-03-25 16:43:15	2026-03-25 16:43:15	husha-0150d64412b543db
2	ash@ash.com	\$2y\$12\$1.446kQ2qODJal40	2026-03-25 16:44:33	2026-04-13 14:31:21	ashlon
3	ast56@gl.edu	\$2y\$12\$btMvAInOcpdfeiU	2026-04-12 21:38:50	2026-04-13 00:20:04	ash
4	toyforashlon54@gmail.com	\$2y\$12\$0sfG0UdalByK4kuZ	2026-04-13 22:33:33	2026-04-13 22:33:33	luis

Users Table



Saved: 4/13/2026 6:34:47 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Progress: 66%

Task #1 (0.67 pts.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP, this is not a valid form, but it is a valid form)

the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
<h3>Login</h3>
<form onsubmit="return validate(this)" method="POST">
  <div>
    <label for="email">Email or Username</label>
    <input id="email" type="text" name="email" required value="<?php echo se($_POST, 'email'); ?>" />
  </div>
  <div>
    <label for="pw">Password</label>
    <input type="password" id="pw" name="password" required minlength="8" />
  </div>
  <input type="submit" value="Login" />
</form>
<script>
```

Login HTML Code Snippet

Login

Email or Username

Password

! Please fill out this field.

Password cannot be empty

Login

Email or Username

Password

! Please fill out this field.

Email cannot be empty

Login

Email or Username

Password

Please lengthen this text to 8 characters or more (you are currently using 4 characters).

Password does not meet length requirements

 Saved: 4/13/2026 7:03:39 PM

⇒ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Similar to registration, this validation is done to ensure that user input meets the required conditions before being processed. This includes checking for empty fields, correct formats, and valid input lengths. This system prevents invalid data from being submitted and allows users to correct errors before continuing.

 Saved: 4/13/2026 7:03:39 PM

≡ Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

📄 Part 1:

Progress: 100%

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

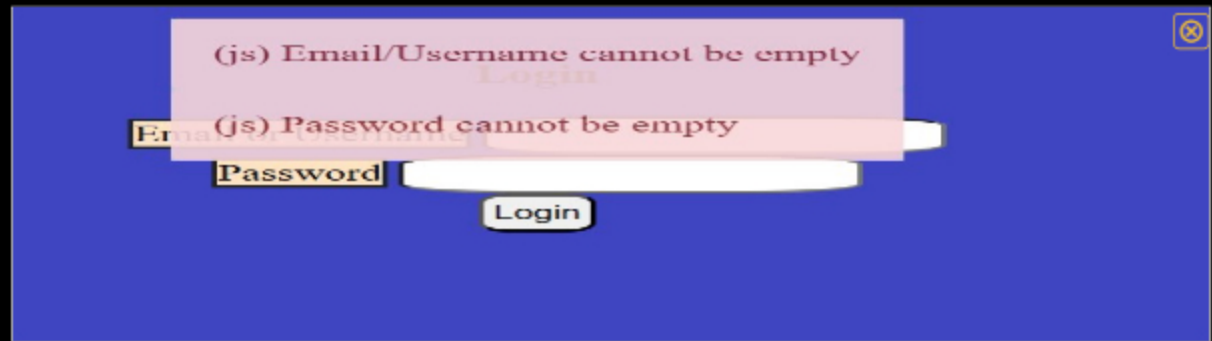
```
<script>
function validate(form) {
  // TODO: Implement JavaScript validation (you'll do this on your own towards the end of Milestone1)
  // ensure it returns false for an error and true for success
  let email = form.email.value;
  let password = form.password.value;

  let isValid = true;

  let flashDiv = document.getElementById("flash");
  if(!flashDiv) flashDiv.innerHTML = "";

  if(!isEmpty(email)){
    flash("Email/Username cannot be empty", "danger");
    isValid = false;
  }
  if(!isEmpty(password)){
    flash("Password cannot be empty", "danger");
    isValid = false;
  }
  return isValid;
}
```

Login JS Validate



The screenshot shows a login form on a blue background. There are two input fields: 'Email or Username' and 'Password'. Above the 'Email or Username' field, a pink error message box says '(js) Email/Username cannot be empty'. Above the 'Password' field, a pink error message box says '(js) Password cannot be empty'. A 'Login' button is located below the password field. The form is surrounded by a dark blue border.

Email and Username Validation Flash Message



Saved: 4/13/2026 7:16:37 PM

≡ Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JS validation is triggered using the onsubmit event, which runs the validate() function and displays error messages while preventing form submission if inputs are invalid.



Saved: 4/13/2026 7:16:37 PM

≡ Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

📌 Part 1:

Progress: 100%

Details:

- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch

(the one that checks against the DB hash)

```
1 <?php
2 // ...
3 // ...
4 // ...
5 // ...
6 // ...
7 // ...
8 // ...
9 // ...
10 // ...
11 // ...
12 // ...
13 // ...
14 // ...
15 // ...
16 // ...
17 // ...
18 // ...
19 // ...
20 // ...
21 // ...
22 // ...
23 // ...
24 // ...
25 // ...
26 // ...
27 // ...
28 // ...
29 // ...
30 // ...
31 // ...
32 // ...
33 // ...
34 // ...
35 // ...
36 // ...
37 // ...
38 // ...
39 // ...
40 // ...
41 // ...
42 // ...
43 // ...
44 // ...
45 // ...
46 // ...
47 // ...
48 // ...
49 // ...
50 // ...
51 // ...
52 // ...
53 // ...
54 // ...
55 // ...
56 // ...
57 // ...
58 // ...
59 // ...
60 // ...
61 // ...
62 // ...
63 // ...
64 // ...
65 // ...
66 // ...
67 // ...
68 // ...
69 // ...
70 // ...
71 // ...
72 // ...
73 // ...
74 // ...
75 // ...
76 // ...
77 // ...
78 // ...
79 // ...
80 // ...
81 // ...
82 // ...
83 // ...
84 // ...
85 // ...
86 // ...
87 // ...
88 // ...
89 // ...
90 // ...
91 // ...
92 // ...
93 // ...
94 // ...
95 // ...
96 // ...
97 // ...
98 // ...
99 // ...
100 // ...
```

Login PHP code snippet

Invalid email address.

Invalid password length.

Email or Username Password Login

Invalid Email and Password length

Invalid login attempt. Please check your email and password.

Email or Username Password Login

Invalid login attempt

Email/Username must not be empty.

Username must be lowercase, alphanumerical, and can only contain _ or -

Password must not be empty.

Password must be at least 8 characters long.

Email or Username Password Login

Empty Username and Password fields

 Saved: 4/13/2026 7:32:05 PM

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The PHP validation is done using conditional statements that check each field against the requirements such as being non-empty, having the correct format, and meeting length requirements. Each validation is performed independently, so multiple error messages can be displayed at the same time. This ensures that all issues are identified and visible to the user.



Saved: 4/13/2026 7:32:05 PM

Task #2 (0.67 pts.) - Related URLs

Progress: 100%

Details:

- Include the direct link to this file from the Milestone branch (should end in `.php` , , zero credit if it does not)
- Include direct link to the file on Render.com Prod (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1[https://github.com/PanicGeneral/ast56-](https://github.com/PanicGeneral/ast56-IT202-008Milestone1/public_html/project/login.php)

URL

<https://github.com/PanicGen>[IT202-008Milestone1/public_html/project/login.php](https://github.com/PanicGeneral/ast56-IT202-008Milestone1/public_html/project/login.php)**URL #2**[https://ast56-](https://ast56-it202-008.onrender.com/project/login.php)

URL

[https://ast56-it202-008.onren](https://ast56-it202-008.onrender.com/project/login.php)[it202-008.onrender.com/project/login.php](https://ast56-it202-008.onrender.com/project/login.php)**URL #3**[https://github.com/PanicGeneral/ast56-](https://github.com/PanicGeneral/ast56-IT202-008/pull/29)

URL

<https://github.com/PanicGen>[IT202-008/pull/29](https://github.com/PanicGeneral/ast56-IT202-008/pull/29)

Saved: 4/13/2026 7:34:29 PM

Task #3 (0.67 pts.) - User Session Evidence

Progress: 0%

Details:

- From the render.com logs (Logs tab), capture the session error_log
- It should show the id, username, email, and roles



Missing Caption

Not saved yet

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Progress: 100%

Task #1 (1 pt.) - Evidence

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the successful logout message (visit the proper file on Render.com qa after a manual deploy)
- Show the code snippet of the logout page

```
public_html > project > logout.php
You, 23 hours ago | 1 author (You)
1 <?php
2 // require functions.php to pull in flash()
3 require( DIR . "/../lib/functions.php");
4 reset_session();
5 flash("You have been logged out","success");
6 header("Location: login.php"); // redirect back to login
```

Logout Code Snippet

You have been logged out

Email or Username

Password

Login

Logout Message



Saved: 4/13/2026 7:45:45 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

<https://github.com/PanicGeneral/ast56-IT2026-0025>



URL

<https://github.com/PanicGeneral/>



Saved: 4/13/2026 7:45:45 PM

Part 3:

Progress: 100%

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The logout code works by calling `reset_session()`, which terminates the session and removes all stored user data. This action logs the user out by clearing their authentication information. A flash message is then displayed to confirm the logout, and the user is redirected to the login page.



Saved: 4/13/2026 7:45:45 PM

Section #4: (2 pts.) Feature: Security Rules

Progress: 100%

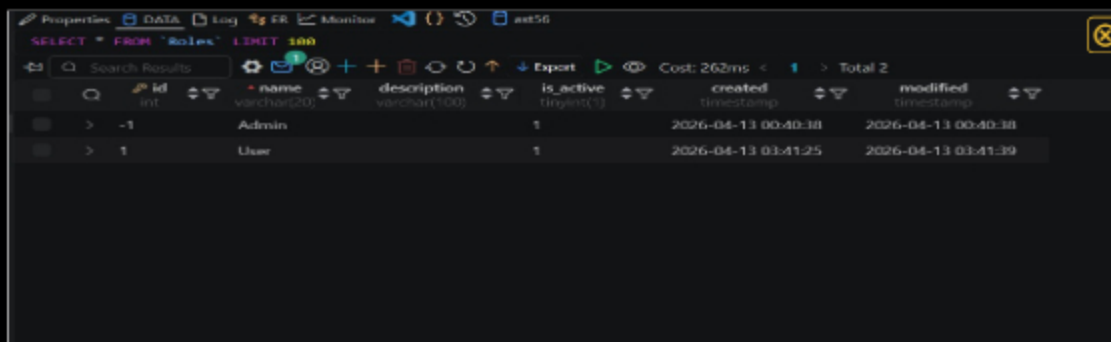


Task #1 (1 pt.) - Database Tables

Progress: 100%

Details:

- From the vs code mysql tool, show both the `Roles` and `UserRoles`

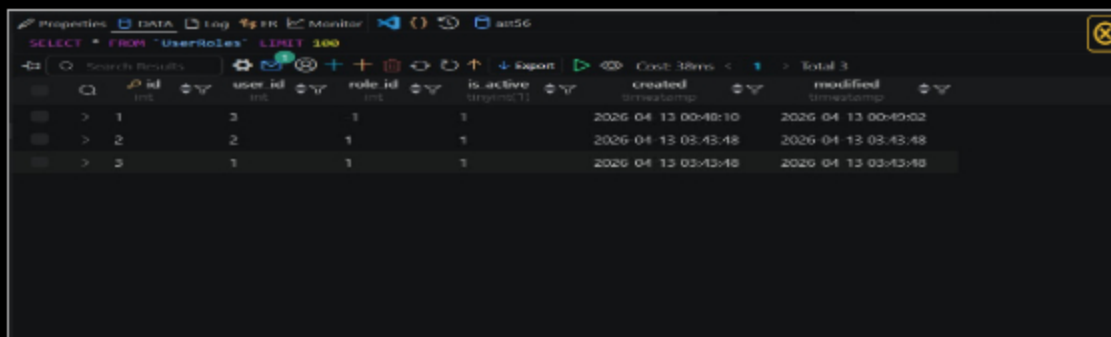


Properties DATA Log ER Monitor ast56

```
SELECT * FROM `Roles` LIMIT 100
```

	id	name	description	is_active	created	modified
	int	varchar(20)	varchar(100)	tinyint(1)	timestamp	timestamp
>	-1	Admin		1	2026-04-13 00:40:38	2026-04-13 00:40:38
>	1	User		1	2026-04-13 03:41:25	2026-04-13 03:41:39

Role Table



Properties DATA Log ER Monitor ast56

```
SELECT * FROM `UserRoles` LIMIT 100
```

	id	user_id	role_id	is_active	created	modified
	int	int	int	tinyint(1)	timestamp	timestamp
>	1	3	1	1	2026-04-13 00:40:30	2026-04-13 00:40:02
>	2	2	1	1	2026-04-13 05:43:48	2026-04-13 03:43:48
>	3	1	1	1	2026-04-13 03:42:48	2026-04-13 03:43:48

UserRoles Table

 Saved: 4/13/2026 7:46:53 PM

≡ Task #2 (1 pt.) - Demo Security Rules

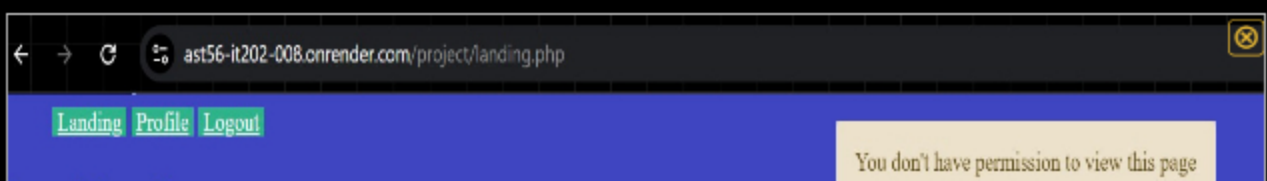
Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the message related to needing to be logged in (i.e., try to manually)
- Show the message related to not having the proper permission/role (i.e.,
- Include a snippet of the login check function
- Include a snippet of the role check function



Landing Page

Welcome, Luis!

Permission Needed

You must be logged in to view this page

Email or Username

Password

Login

Must Be logged in

```
1 <?php
2 //note we need to go up 1 more directory
3 require(__DIR__ . "../..../partials/nav.php");
4
5 if (!has_role("Admin")) {
6     flash("You don't have permission to view this page", "warning");
7     die(header("Location: " . get_url("landing.php")));
8 }
9
```

Code Snippet for Permission

```
function is_logged_in($redirect = false, $destination = "login.php")
{
    $isLoggedIn = isset($_SESSION["user"]);
    if ($redirect && !$isLoggedIn) {
        //if this triggers, the calling script won't receive a reply since die()/exit() terminates it
        flash("You must be logged in to view this page", "warning");
        $path = $destination;
        // handle relative paths
    }
}
```

Must Be logged in Code Snippet



Saved: 4/13/2026 9:46:39 PM

Part 2:

Progress: 100%

Details:

- Include the pull request url for this feature

URL #1

URL



Saved: 4/13/2026 9:46:39 PM

≡ Part 3:

Progress: 100%

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

The login check works by verifying whether a user session exists. The `is_logged_in()` function checks if the "user" key is set in the session. If it is not set, the system displays a flash message and redirects the user to the login page.

The role check works by verifying whether the logged-in user has a specific role stored in their session. The `has_role()` function loops through the user's assigned roles and checks for a match. If the user does not have the required role, a flash message is displayed and the user is redirected.



Saved: 4/13/2026 9:46:39 PM

Section #5: (2 pts.) Feature: User Profile

Progress: 100%

≡ Task #1 (1 pt.) - Validation

Progress: 100%

Details:

- Show the relevant code snippets for each validation layer (html, js, php)
- Show the relevant demo of each validation layer (html, js, php)
- Briefly explain how the validation steps work at each layer

≡ Task #1 (0.33 pts.) - HTML Form/Validation

Progress: 100%

Details:

- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)

```
//
<script src="/js/">
<form method="POST" onsubmit="return validate(this);">
  <div class="mb-3">
    <label for="email">Email</label>
    <input type="email" name="email" id="email" value="{{2php: $_($email); 4x}}"/>
  </div>
  <div class="mb-3">
    <label for="username">Username</label>
    <input type="text" name="username" id="username" value="{{2php: $_($username); 2x}}"/>
  </div>
  <div class="mb-3">
    <label for="cp">Current Password</label>
    <input type="password" name="currentpassword" id="cp"/>
  </div>
  <div class="mb-3">
    <label for="np">New Password</label>
    <input type="password" name="newpassword" id="np"/>
  </div>
  <div class="mb-3">
    <label for="comp">Confirm Password</label>
    <input type="password" name="confirmpassword" id="comp"/>
  </div>
  <input type="submit" value="Update Profile" name="save"/>
</form>
</script>
```

HTML code snippet

Profile

Email

Username

Current Password

New Password

Confirm Password

Update Profile

Please fill out this field.

Fill out email

Profile

Email

Username

Current Password

New Password

Please lengthen this text to 8 characters or more (you are currently using 4 characters).

Invalid Password Length

Saved: 4/13/2026 11:23:14 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The HTML validation uses built-in browser features such as the required attribute, type="email" to ensure that user input meets basic requirements before the form is submitted. These validations check for empty fields and proper email formatting helping prevent invalid data from being entered.



Saved: 4/13/2026 11:23:14 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
function validateForm() {
  let form = document.querySelector('form');
  let email = form.querySelector('#email');
  let username = form.querySelector('#username');
  let password = form.querySelector('#password');
  let confirmPassword = form.querySelector('#confirm-password');

  // Check if any field is empty
  if (email.value === '' || username.value === '' || password.value === '' || confirmPassword.value === '') {
    alert('All fields are required');
    return false;
  }

  // Check email format
  if (!email.value.match(/^[a-zA-Z0-9._-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,4}$/)) {
    alert('Email is invalid');
    return false;
  }

  // Check password length
  if (password.value.length < 8) {
    alert('Password must be at least 8 characters');
    return false;
  }

  // Check password match
  if (password.value !== confirmPassword.value) {
    alert('Passwords do not match');
    return false;
  }

  // If all checks pass, submit the form
  form.submit();
}
```

JS code snippet

(js) Password must be at least 8 characters

(js) Email is invalid

(js) Username is invalid

Current Password

New Password

Confirm Password

Update Profile



Saved: 4/13/2026 11:38:06 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The JavaScript validation is done using a `validate()` function that runs when the form is submitted. It checks conditions such as matching passwords, valid email format, non-empty fields, and proper username formatting. If validation fails, flash messages are displayed and the form is not submitted. This ensures users receive immediate feedback and can correct errors before the data is sent.



Saved: 4/13/2026 11:38:06 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call)

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots) (visit the proper file on Render.com qa after a manual deploy)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the `validate()` function or use the provided helper script in the instructions

```

<?php
$user_id = get_user_id(); // get id from session
$email = get_user_email(); // get email from session
$username = get_username(); // get username from session

// handle email/username update
if (isset($_POST['email'], $_POST['username'])) {
    $new_email = $_POST['email'];
    $new_username = $_POST['username'];
    $username = $_POST['username'];
    $email = $_POST['email'];
    $password = $_POST['password'];
    $confirm_password = $_POST['confirm_password'];

    // validate format
    if (empty($new_email)) {
        $flash["Email must not be empty."] = "danger";
        $flash["Email must not be empty."] = "danger";
        $flash["Email must not be empty."] = "danger";
    }

    // sanitize and validate email
    $new_email = sanitize_email($new_email);
    if (!is_valid_email($new_email)) {
        $flash["Invalid email address."] = "danger";
        $flash["Invalid email address."] = "danger";
        $flash["Invalid email address."] = "danger";
    }
}

```

Section #6: (1 pt.) Misc

necessary

Projects • ast56-IT202-008

10 hrs 55 mins over the **Last 7 Days** in ast56-IT202-008 under **all** branches. 🌲

Overall

[illegible]

Individual work



Saved: 4/13/2026 11:51:34 PM

≡ Task #3 (0.33 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

I learned a lot of things like how to create flash messages, store user data in databases and also how to assign roles on a website.



Saved: 4/13/2026 11:27:02 PM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of this assignemnt as changing the style of the webpage.



Saved: 4/13/2026 11:27:21 PM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was trying to get the permission warning because i kept running into an error when i manually tried to go to a page while not logged in.



Saved: 4/13/2026 11:28:09 PM